

Securing AI Agents on Kubernetes

Identity, Sandboxes, and MCP Tool Governance

Dr. Roland Huß

Distinguished Engineer, Red Hat



Dr. Roland Huß

Distinguished Engineer, Red Hat AI
ro14nd.de

Building a platform for running agentic applications
securely on Kubernetes

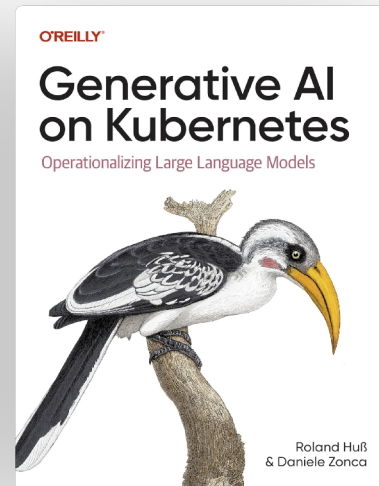
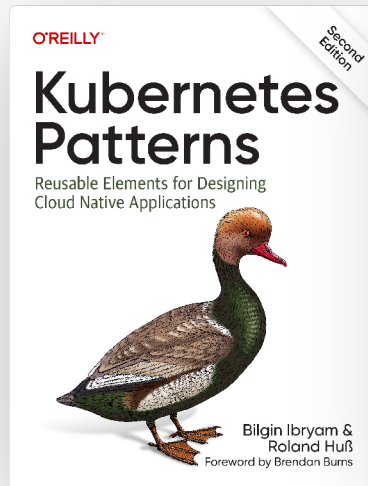
OpenShell contributor · Agentic AI Foundation
Ambassador

30+ years of writing code



Generative AI on Kubernetes

Free download from Red Hat
bit.ly/genaik8s-redhat-free



Agents break every control you already have

Decides at runtime

The call graph does not exist until inference happens.

breaks **deploy-time controls**
NetworkPolicy, RBAC, egress allowlists

One channel for both

Instructions and data are the same tokens in the same window.

breaks **input validation**
there is no prepared statement here

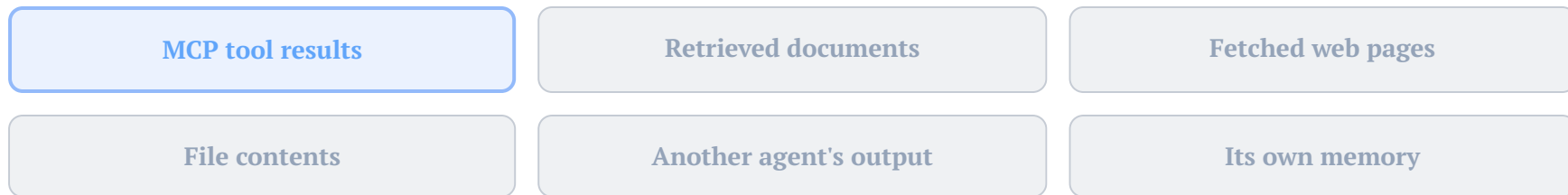
Spans domains by design

It is useful because it crosses systems, so its authority cannot be task-scoped.

breaks **least privilege**
every credential live, every turn

A microservice has a fixed API contract.
An agent has a **probability distribution**.

Everything the agent reads is an instruction channel



All legitimate. None removable. **One context window.**

MCP is not the most dangerous channel.
It is the only one with a **standard**: enforce once, holds everywhere.
And the other channels can be moved **behind** it.

You cannot trust the decision.
Bound the reach. Prove the caller.

Isolation is the foundation, not one layer among three

Policy

What may it **ask for**?

OPA, MCP tool rules

Identity

Who is it, **on whose behalf**?

SPIFFE, OIDC, token exchange

Isolation

What can it **reach at all**?

kernel / hypervisor

Everything above the sandbox is **advisory** if the sandbox leaks.

So we build it in that order.

Policy

What may it ask for?

Identity

Who is it, on whose behalf?

Isolation

What can it **reach at all?** kernel / hypervisor

Bound the reach

Everyone serious landed on the strong end

One axis: **where does the syscall boundary sit, and what does an escape cost?**

OS primitives

fence the process, share the kernel

Anthropic `srt`
OpenAI Codex CLI

Container + hardening

own namespaces, still the host kernel

NVIDIA OpenShell
+ policy proxy

gVisor

a second kernel, in userspace, answers the syscalls

Agent Substrate

microVM / Kata

its own kernel, behind hardware virtualization

Docker Sandboxes
E2B, Vercel, Modal

Full VM

same boundary, more surface

nobody, for agents

weaker

stronger

Convergence, not opinion

Independent teams, same conclusion. The trade is strength against **start latency and density**.

And they compose

`agent-sandbox` is not on this axis: it *picks* a runtime via `RuntimeClass`. OpenShell on Kubernetes runs **on** it, not against it.

For an agent, untrusted code is the product

Every other workload

Running untrusted code is an **accident**.

A bug, a bad dependency, an injection flaw.
Something went wrong.

A coding agent

Running untrusted code is the **feature**.

It executes generated code by design. Nothing
went wrong.

Prompt injection closes the loop: the agent reads a web page, an issue comment, a tool response,
and **that content is now steering it**.

The threat model is not "a bug lets attacker code run".
It is "**attacker code runs, as intended, every time**".

OpenShell: sandboxed agents, policy as YAML

What it is

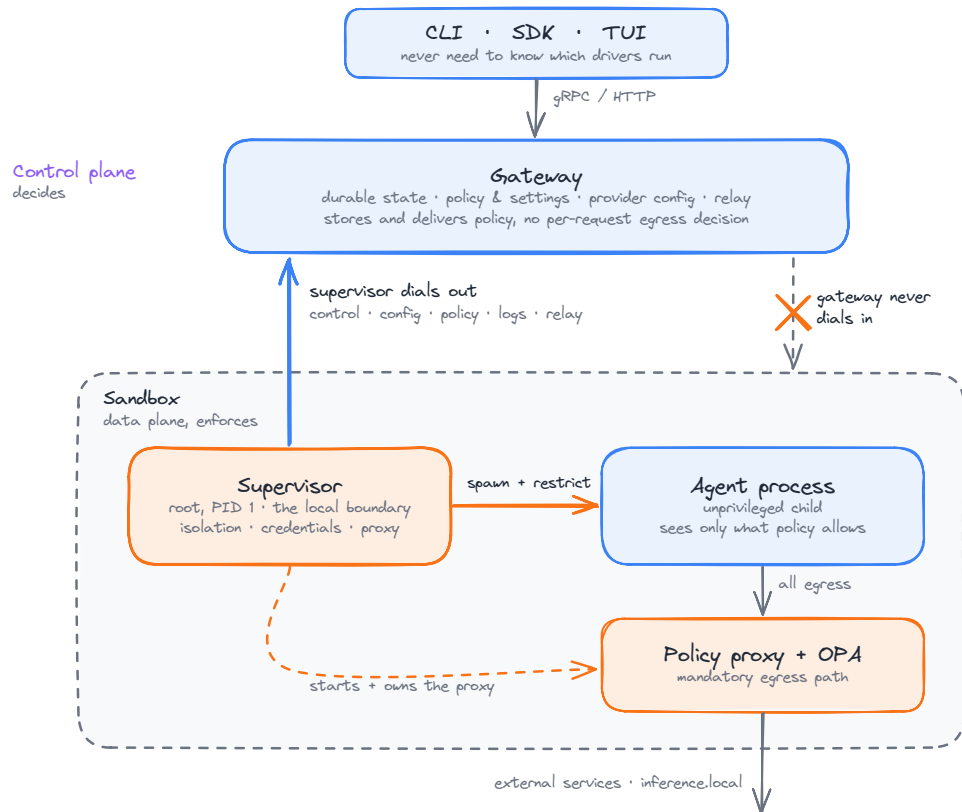
- Agent runtime, Apache 2.0, Rust
- NVIDIA-created; Red Hat, Canonical, Cisco
- Declarative YAML policy
- Filesystem, egress, credentials
- Workspaces, on the way to multi-player
- CNCF Sandbox application filed

How you drive it

- CLI and TUI for humans
- SDKs: Rust, Python, Go, TypeScript
- One gateway API, same protobufs
- Web dashboard in development
- Runs on Kubernetes, Docker, Podman, MicroVM

```
$ openshell sandbox create -- claude
```

Control plane decides, data plane enforces



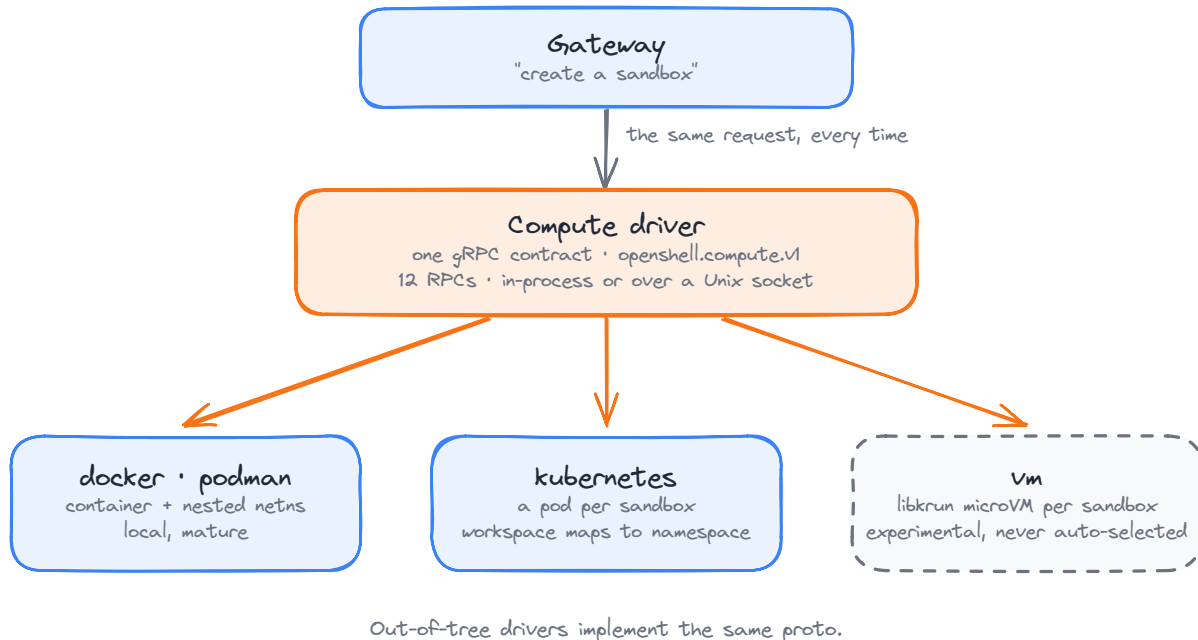
The gateway stores and delivers policy. It makes no per-request egress decision.

Enforcement happens where process identity, filesystem access, and network egress are actually visible: inside the sandbox.

The supervisor dials out. The gateway never dials in.

No compute driver has to solve inbound reachability: no pod IPs, no NAT traversal, no bespoke tunnels.

Same sandbox, different substrate



gRPC, not a plugin API

service `ComputeDriver` , 12 RPCs. Same proto whether the driver is linked in or runs as a separate binary.

No Exec RPC

Lifecycle only. Exec and file sync ride the supervisor's *outbound* stream. The gateway never dials in.

Four driver boundaries, not one

Boundary	Pluggable into
Compute driver	Docker / Podman / Kubernetes / VM · <code>compute_driver.proto</code>
Credentials driver	Vault / Kubernetes Secrets / local encrypted store · <code>credential_driver.proto</code>
Control-plane identity	mTLS / OIDC / local identity
Sandbox identity	SPIFFE / gateway-issued workload identity

“OpenShell should integrate with infrastructure ecosystems **instead of replacing them.**”

OpenShell architecture docs

The kernel enforces what the agent cannot bypass

Its own network stack

One virtual wire out of the sandbox. The proxy owns the other end, so ignoring `HTTP_PROXY` changes nothing.

Filesystem limits the kernel keeps

Sealed at startup by Landlock. The agent can narrow them, never widen them.

An allowlist of system calls

seccomp drops everything else at the kernel boundary. No debugger, no mounts, no raw sockets.

The agent is never root

The supervisor keeps the privileges. The agent drops them before a line of model output runs.

Filesystem and process limits are sealed. Network rules can be reloaded live. Neither can be relaxed by the agent.

It checks *which binary* opened the connection

```
1  network_policies:
2    - endpoint: api.weather.com
3      port: 443
4      binaries:
5        - path: /usr/bin/python3
6          sha256: "a1b2c3..."      # verified on first use
7      methods: [GET, POST]
```

How

- `/proc/<pid>/exe` — kernel-trusted, not `argv[0]`
- Walks the process tree for ancestors
- SHA256 on first use; mismatch denies instantly

Why

- The agent shells out to `curl` → **denied**
- Any other binary, same destination → **denied**
- Endpoint + port + method + binary must *all* match

If no rule matches, the request is **denied**.

Policy

What may it ask for?

Identity

Who is it, **on whose behalf?** SPIFFE, OIDC, token exchange

Isolation

What can it reach at all?

✓ built

Prove the caller

Three answers to "who is calling?"

Impersonation

The user's token is passed straight through.

The agent **disappears** from the audit trail. Your log says the user did it.

Service account

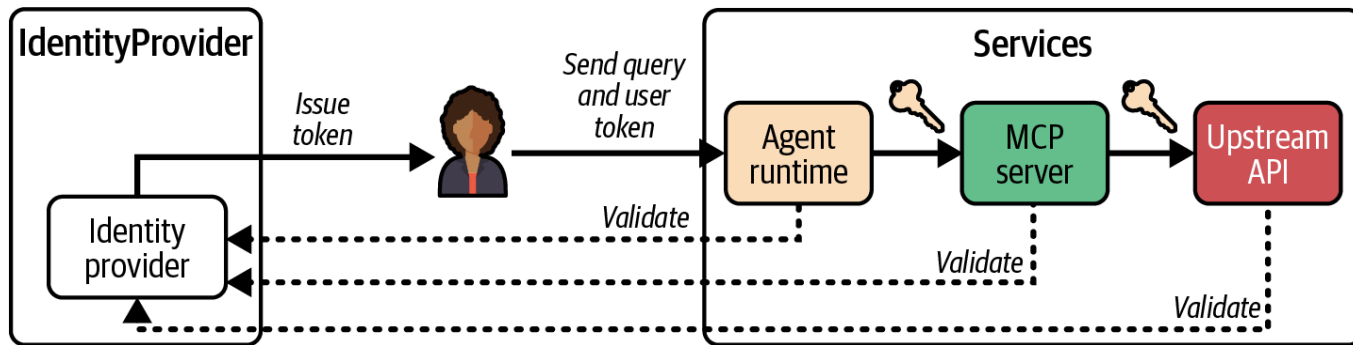
The agent uses its own identity.

Per-user attribution is **lost**. The server trusts the union of every user's permissions.

Delegation (RFC 8693)

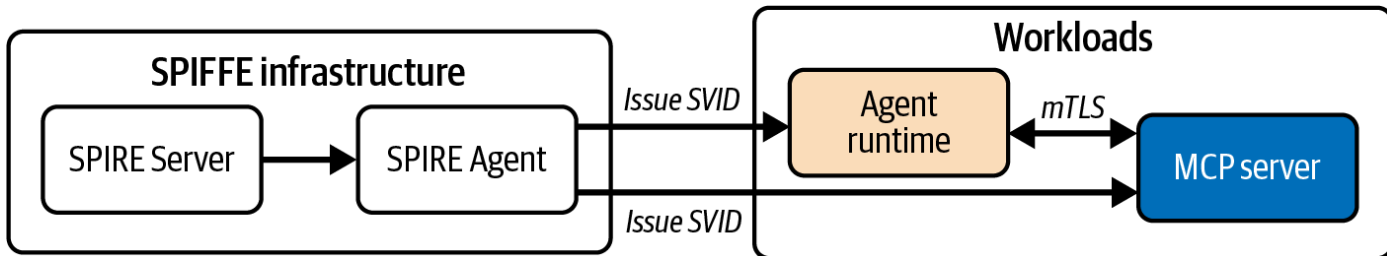
One token carries both.

sub = the user, act = the agent. The server enforces the **intersection**.



Impersonation. From *Generative AI on Kubernetes* (O'Reilly)

Workload identity, not API keys



From *Generative AI on Kubernetes* (O'Reilly)

Shared API keys

- Long-lived, copied everywhere
- Readable from env vars and logs
- Revoke it for one, revoke it for all

A short-lived identity per workload

SPIFFE calls it an SVID

- Expires in minutes, rotates itself
- Only issued after the platform checks what you are
- Revocable one workload at a time

The supervisor holds the identity, never the agent. It fetches it inside the sandbox, on the proxy path.

A workload identity is earned, not issued

1. Node attestation

The agent on each node proves **which node** it is, using a token the cluster signs.

2. Workload attestation

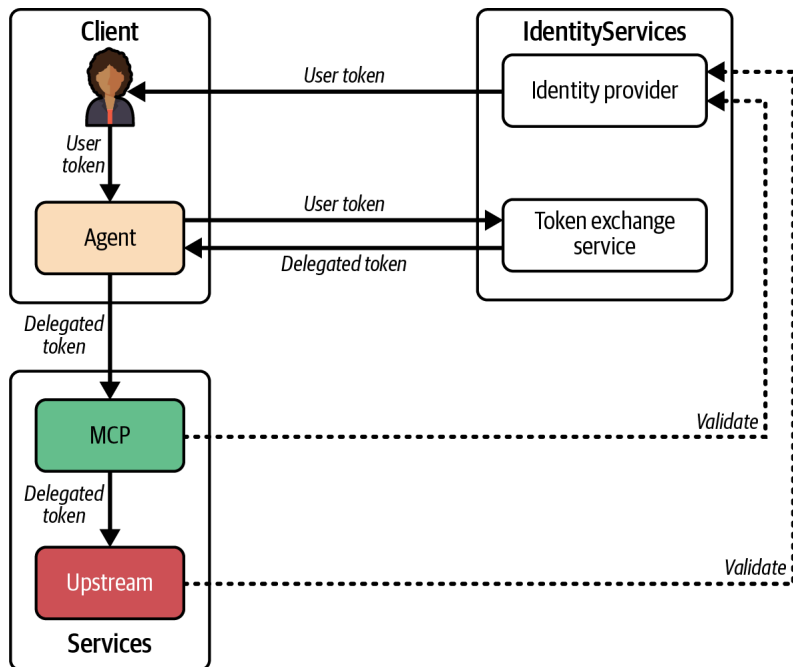
Before handing over an identity, the kernel is asked what the calling process **actually is**.

What it checks, none of which the process gets to assert about itself

`k8s:ns · k8s:sa · k8s:pod-label · k8s:container-image`

This is the difference from a secret. An API key is trust you can copy. This is a conclusion the platform reaches about a running process, and re-reaches every few minutes.

Delegation keeps both identities in one token



From *Generative AI on Kubernetes* (O'Reilly)

`client_credentials`

One identity: the agent's own. The human who asked has **disappeared**, so the server trusts the **union** of every permission any user might need.

On-Behalf-Of (RFC 8693)

Both identities in one token: `sub` = the user, `act` = the agent.
`act` **needs**, so handoffs stay readable, and the server enforces the **intersection**.

Inside the exchange

```
1  payload = {  
2    "grant_type":  
3      "urn:ietf:params:oauth:grant-type:token-exchange",  
4    "subject_token": user_token,      # who it is for  
5    "actor_token":   agent_token,     # who is acting  
6    "audience":     mcp_server,      # where it is valid  
7  }
```

The issued JWT: `sub` = the user, `act` = the agent.

`act` **neests**, so a multi-agent handoff reads as a chain instead of a mystery.

Two exchange points, one mechanism

The agent runtime exchanges for an MCP-targeted token. The MCP server can exchange again for an upstream-targeted one. Only `act` changes.

What it buys the resource server

A composite rule it can actually evaluate:
*allow if the user has permission **and** the agent is authorized for this operation.*

The honest cost

A token service you now run, in front of every tool call. Its failures are your failures. Cache it, or pay a round trip per call.

Policy

What may it **ask for**? OPA, MCP tool rules

Identity

Who is it, on whose behalf?

✓ built

Isolation

What can it reach at all?

✓ built

Decide what it may ask for

The tool name lives inside the JSON-RPC body

```
1 POST /mcp → { "jsonrpc": "2.0", "method": "tools/call",  
2               "params": { "name": "database_query", ... } }
```

At layer 4

One host. One port. One TLS session.

Every tool call looks identical.

At layer 7

weather_lookup → **allowed**

database_query → **denied**

A denial is not a dead end. The agent gets structured `next_steps` and can propose a policy change.

The agent holds a placeholder, not a secret

```
1 # inside the sandbox, this is literally what the agent sees
2 GITHUB_TOKEN=openshell:resolve:env:v3_GITHUB_TOKEN
```

The real value is spliced in **per request, inside the proxy**, and only when the destination matches.

The agent, not the sandbox

The **agent process** never holds it. The supervisor does, in a different process.

One escape hatch

`allow_uninspected_credentials` passes real credentials through TLS the proxy cannot read.

Neither half works alone. Isolation without identity is unaccountable. Identity without isolation is bypassable.

Demo

Identity — the sandbox gets an SVID. The agent cannot read it.

Delegation — SVID exchanged for a token carrying `sub` and `act` .

Policy — `weather_lookup` passes L7 inspection. `database_query` does not.

Isolation — the agent shells out to `curl` to go around it. Per-binary policy blocks it.

And it cannot route around any of it.

The standards are not settled. Mind the maturity gap.

Standard	What it is	Status
OAuth 2.0 + OIDC	Delegated authorization and login	✅ RFC 6749, OIDC Core 1.0
Token exchange	Both identities in one token	✅ RFC 8693, 2020
IETF WIMSE	Workload identity: identifier URI, creds, proof token	adopted WG items
MCP authorization	Auth for the MCP transport itself	revision 2026-07-28
AAuth	Agent auth via HTTP Message Signatures	individual draft, no WG
Cross-App Access	Identity Assertion JWT Authorization Grant	draft-04
OIDF AuthZEN	COAZ: MCP tool authorization profile	WG drafts

The two ✅ rows are boring, deployed, and enough to ship today. Below them, **WIMSE** has adopted WG documents and **AAuth** is one person's Internet-Draft.

The sandbox is the part that does not depend on **which spec wins**.

Different layers stop different attacks

Attack	Layer	What stops it
Data exfiltration via <code>curl</code>	Isolation	the kernel names the binary, and curl is not on the list
Lateral move to another MCP server	Isolation	one wire out, through the proxy, default deny
Stolen token replayed	Identity	the token is bound to a key the thief does not have
Prompt injection to an unauthorized tool	Policy	the proxy reads the tool name and refuses

Every one of these is stopped **below** the agent.
None depend on the agent behaving, the prompt holding, or the model refusing.

Thank You!

Put the controls in the environment, not in the prompt.

Every link from this talk



bit.ly/agent-sec-2026

Connect

✉ rhuss@redhat.com

in linkedin.com/in/ro14nd

🌐 ro14nd.de

Questions?

On Kubernetes, the layers interact

It runs *on* agent-sandbox

Creates the upstream `agents.x-k8s.io` `Sandbox` CR. The agent-sandbox controller creates the pod.

OpenShell owns **no CRDs of its own** and creates no Pods directly.

You can put Kata underneath

Sets `runtimeClassName` on the pod spec.

```
1  server:
2    defaultRuntimeClassName: kata-qemu
```

But gVisor is explicitly not supported, and that is the interesting part

Sidecar topology needs pod-local nftables gVisor won't give the init container. And gVisor's sentry provides **no Landlock**, so OpenShell's filesystem policy *degrades* under it.

"Runtime classes do not re-enable the OpenShell privilege-drop or supervisor mount-isolation controls that sidecar mode relaxes. Use them as an **additional workload boundary, not as a replacement.**"

Backup: Helm and deployment topology

Deployment model

- Gateway runs as StatefulSet (SQLite) or Deployment (external DB)
- Gateway creates the upstream `agents.x-k8s.io` `Sandbox` CR; the agent-sandbox controller schedules the Pod
- Version negotiation: detects the served API, prefers `v1beta1`, falls back to `v1alpha1`

Two chart ownership boundaries

- *Gateway chart*: gateway workload, config, Services, PKI, cluster-scoped resources
- *Workspace chart*: installed into a pre-provisioned sandbox namespace; owns only the sandbox ServiceAccount, namespaced RBAC, and ingress NetworkPolicy
- Its RoleBinding names the gateway ServiceAccount explicitly, so the two releases have disjoint lifecycles
- Shared mode targets one external namespace; operator mode maps workspace names to many

RuntimeClass

- `server.defaultRuntimeClassName` in Helm values; per-sandbox override via `driver_config`
- Kata validated (CI pins `kata-qemu`). gVisor unsupported: sidecar mode needs pod-local nftables, and gVisor's sentry provides no Landlock

Agent Sandbox CRDs and controller lifecycle remain operator-owned — the chart does not install them.

Backup: Supply-Chain Provenance with Sigstore

The problem: How do you trust the container image running inside the sandbox?

rossoctl + SPIRE + Sigstore integration:

- SPIRE's Kubernetes workload attestor has built-in Sigstore support
- When enabled: SPIRE **refuses to issue identity** to workloads running unsigned images
- An agent with a tampered image cannot even obtain a SPIFFE identity

Audit mode for gradual rollout:

```
1  --sigstore-audit-mode=true
```

- Evaluates every bundle but **never blocks**
- Verification failures: logged, recorded as K8s events, reflected in AgentCard status
- Full visibility into what would fail under enforcement
- Switch to enforcement when confident

Result: identity issuance becomes a supply-chain gate. No signed image, no identity, no access.

Backup: Gateway Scope Enforcement

8 fine-grained scopes

Scope	Controls
<code>sandbox:read</code>	List/get sandboxes
<code>sandbox:write</code>	Create/delete sandboxes
<code>provider:read</code>	List/get providers
<code>provider:write</code>	Create/update/delete providers
<code>config:read</code>	Read gateway config
<code>config:write</code>	Modify gateway config
<code>inference:read</code>	Query inference routing
<code>inference:write</code>	Modify inference routing

Token format compatibility

- Space-delimited (Keycloak, Entra ID)
`scope: "sandbox:read sandbox:write"`
- JSON arrays (Okta)
`scp: ["sandbox:read", "sandbox:write"]`
- Standard OIDC scopes (`openid` , `profile` , `email`) ignored for authorization

Modes

- `auth-only` — any valid JWT accepted
- `RBAC` — admin/user roles plus scopes

Plus `openshell:all` — wildcard over all scoped methods.

Backup: the policy advisor loop

Two proposers, one referee. A denial starts a negotiation instead of ending one.

Proposers

- **Mechanistic:** sandbox-side mapper turns an observed denial into a draft rule
- **Agent-authored:** via `policy.local`

Agent-authored chunks **cannot** request `protocol: tcp` or `tls: skip` — rejected sandbox-side *and* re-checked at the gateway.

Referee: the gateway

Canonicalizes against live policy, then runs the **same** merge, validation, provider composition, credential preflight and **prover** evaluation the real change would hit.

Provider rules are immutable inputs; the sandbox contribution is stored as an overlay.

Auto-approval requires all three: operator opted in (`proposal_approval_mode: auto`), prover delta empty, and recomputed security notes empty. Default at both scopes is `manual` . Audit event says "auto-approved: no new prover findings" — never `safe` .

Backup: what most deployments get wrong

Prompt-level guardrails. The agent can be told to ignore them. Untrusted input is the attack surface *and* the feature.

Shared API keys. No identity, no attribution, no revocation granularity.

Network allow/deny without L7. Blocks the server, not the tool. Every MCP call looks the same at layer 4.

No delegation model. Either the agent holds the user's full credentials, or the user vanishes from the audit trail.

Treating the sandbox as optional hardening. It is the foundation. The other four only hold if it does.

Backup: one agent secured. What about a hundred?

OpenShell secures the **individual sandbox**. The fleet needs more.

agentgateway

Linux Foundation

One proxy in front of every MCP server. Rule-based access control, built on Gateway API.

kagent

Solo.io

Agents as first-class Kubernetes objects. Tracing and service-mesh identity included.

rossoctl

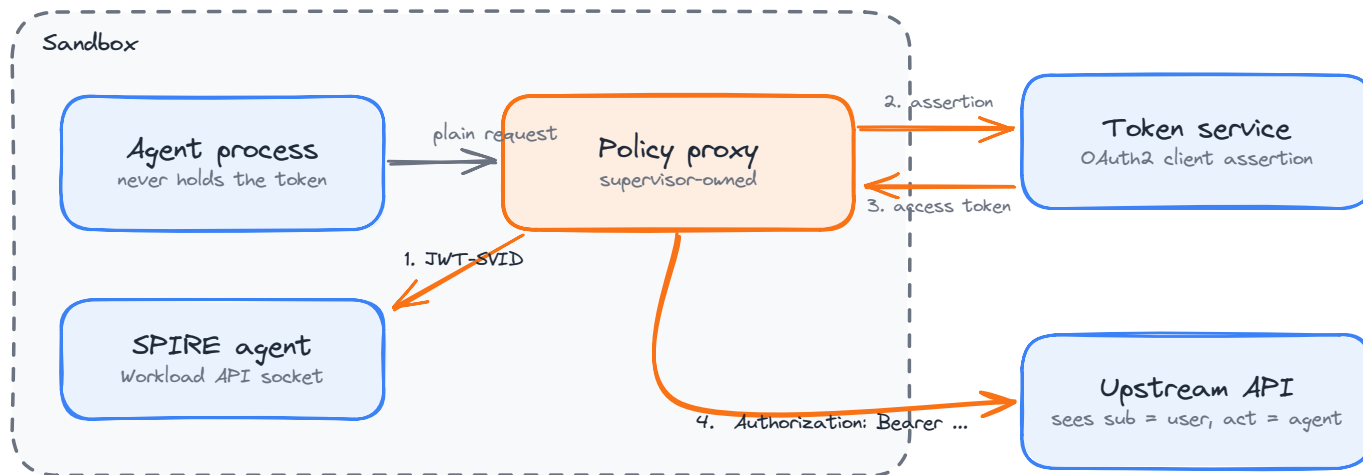
Red Hat

A sidecar that does the token exchange for you, wired to workload identity and signing.

Sandbox, network governance, lifecycle, platform identity.

Complementary layers, not competitors.

Backup: SVID in, header out, the agent stays blind



The credential is added on the way out. The agent never sees it.

This is where the two layers stop being separate topics. **None of this works if the agent can go around the proxy**, and that is per-binary enforcement.